

Anime Analytics — MySQL & SQL

Project Overview

Anime Analytics is a mini data analytics project developed using **MySQL and SQL**.

The project creates a relational anime database containing information about:

- Anime titles
- Genres
- Animation studios
- Characters
- Users
- Reviews
- Ratings
- Episodes
- Character popularity
- Character power levels

SQL queries are used to perform data exploration, filtering, aggregation, joins, subqueries, CTE analysis, and ranking.

Project Objectives

1. Create a structured anime database using MySQL.
2. Store anime, genre, studio, character, user, and review information.
3. Establish relationships between different tables.
4. Perform SQL-based data analysis.
5. Analyze anime ratings and episode counts.
6. Analyze anime genres and studios.
7. Analyze character popularity and power levels.
8. Analyze user reviews and scores.
9. Practice advanced SQL concepts.
10. Generate useful analytical results from relational data.

Technologies Used

Technology	Purpose
MySQL	Database creation and management
SQL	Data querying and analysis
MySQL Workbench	SQL development and execution
GitHub	Project documentation and version control

Database Name

AnimeAnalytics

Database creation:

```
DROP DATABASE IF EXISTS AnimeAnalytics;
```

```
CREATE DATABASE AnimeAnalytics;
```

```
USE AnimeAnalytics;
```

The database can therefore be recreated and executed from the beginning.

Database Structure

The database contains six main tables:

1. genre
 2. studio
 3. anime
 4. characters
 5. users
 6. reviews
-

Dataset Summary

Data	Records
------	---------

Genres	10
--------	----

Studios	10
---------	----

Anime	20
-------	----

Characters	50
------------	----

Users	8
-------	---

Reviews	10
---------	----

SQL Operations Performed

1. Basic Data Exploration

```
SELECT * FROM anime;
```

```
SELECT * FROM characters;
```

Used to view and verify stored data.

2. Anime Rating Analysis

```
SELECT anime_name, rating
```

```
FROM anime;
```

Highly rated anime:

```
SELECT anime_name, rating
```

```
FROM anime
```

```
WHERE rating >= 8.5;
```

3. Episode Analysis

```
SELECT anime_name, episodes
```

```
FROM anime
```

```
ORDER BY episodes DESC;
```

Used to find anime with the highest number of episodes.

4. Recent Anime Analysis

```
SELECT anime_name, release_year  
FROM anime  
WHERE release_year >= 2020;
```

Aggregate Analysis

Total Anime

```
SELECT COUNT(*) AS total_anime  
FROM anime;
```

Average Rating

```
SELECT AVG(rating) AS average_rating  
FROM anime;
```

Highest Rating

```
SELECT MAX(rating) AS highest_rating  
FROM anime;
```

Lowest Rating

```
SELECT MIN(rating) AS lowest_rating  
FROM anime;
```

Total Episodes

```
SELECT SUM(epsisodes) AS total_episodes  
FROM anime;
```

Genre Analysis

```
SELECT  
    g.genre_name,  
    COUNT(a.anime_id) AS total_anime,
```

```
    AVG(a.rating) AS average_rating
FROM genre g
LEFT JOIN anime a
ON g.genre_id = a.genre_id
GROUP BY g.genre_name
ORDER BY total_anime DESC;
```

This analyzes:

- Number of anime by genre
 - Average rating by genre
 - Genre distribution
-

Studio Analysis

```
SELECT
    s.studio_name,
    COUNT(a.anime_id) AS total_anime,
    AVG(a.rating) AS average_rating
FROM studio s
LEFT JOIN anime a
ON s.studio_id = a.studio_id
GROUP BY s.studio_name
ORDER BY average_rating DESC;
```

This analyzes:

- Anime count by studio
 - Average rating by studio
-

Anime + Genre + Studio

```
SELECT
    a.anime_name,
```

```
    g.genre_name,  
    s.studio_name,  
    a.episodes,  
    a.rating,  
    a.release_year  
FROM anime a  
JOIN genre g  
ON a.genre_id = g.genre_id  
JOIN studio s  
ON a.studio_id = s.studio_id  
ORDER BY a.rating DESC;
```

This combines information from three related tables.

Character Analysis

Character Popularity

```
SELECT  
    character_name,  
    popularity  
FROM characters  
ORDER BY popularity DESC;
```

High Power Characters

```
SELECT  
    character_name,  
    power_level  
FROM characters  
WHERE power_level >= 95  
ORDER BY power_level DESC;
```

Character Role Analysis

```
SELECT
    role_type,
    COUNT(*) AS total_characters,
    AVG(popularity) AS average_popularity
FROM characters
GROUP BY role_type;
```

This compares:

- Main characters
 - Supporting characters
 - Character count
 - Average popularity
-

Anime and Character JOIN

```
SELECT
    a.anime_name,
    c.character_name,
    c.role_type,
    c.power_level,
    c.popularity
FROM anime a
JOIN characters c
ON a.anime_id = c.anime_id
ORDER BY a.anime_name, c.popularity DESC;
```

Popular Characters

```
SELECT
    a.anime_name,
```

```
    c.character_name,  
    c.popularity  
FROM anime a  
JOIN characters c  
ON a.anime_id = c.anime_id  
WHERE c.popularity >= 95  
ORDER BY c.popularity DESC;
```

Character Count by Anime

```
SELECT  
    a.anime_name,  
    COUNT(c.character_id) AS character_count  
FROM anime a  
LEFT JOIN characters c  
ON a.anime_id = c.anime_id  
GROUP BY a.anime_name  
ORDER BY character_count DESC;
```

Rating Categories Using CASE

```
SELECT  
    anime_name,  
    rating,  
    CASE  
        WHEN rating >= 9 THEN 'Excellent'  
        WHEN rating >= 8 THEN 'Very Good'  
        WHEN rating >= 7 THEN 'Good'  
        ELSE 'Average'  
    END AS rating_category
```

FROM anime;

Categories

Rating	Category
--------	----------

9.0+	Excellent
------	-----------

8.0–8.9	Very Good
---------	-----------

7.0–7.9	Good
---------	------

Below 7	Average
---------	---------

Subquery Analysis

Find anime with a rating above the overall average:

```
SELECT
```

```
    anime_name,
```

```
    rating
```

```
FROM anime
```

```
WHERE rating > (
```

```
    SELECT AVG(rating)
```

```
    FROM anime
```

```
)
```

```
ORDER BY rating DESC;
```

The inner query calculates the average rating, while the outer query finds anime above that average.

Top 10 Characters

```
SELECT
```

```
    character_name,
```

```
    power_level,
```

```
    popularity
```

```
FROM characters  
  
ORDER BY popularity DESC  
  
LIMIT 10;
```

★ Review Analysis

```
SELECT  
  
    a.anime_name,  
  
    COUNT(r.review_id) AS total_reviews,  
  
    AVG(r.score) AS average_score  
  
FROM anime a  
  
LEFT JOIN reviews r  
  
ON a.anime_id = r.anime_id  
  
GROUP BY a.anime_name  
  
ORDER BY average_score DESC;
```

This analyzes:

- Total reviews
 - Average review scores
 - Reviews by anime
-

💻 User Review Analysis

```
SELECT  
  
    u.user_name,  
  
    COUNT(r.review_id) AS reviews_given,  
  
    AVG(r.score) AS average_score  
  
FROM users u  
  
LEFT JOIN reviews r  
  
ON u.user_id = r.user_id  
  
GROUP BY u.user_name
```

```
ORDER BY reviews_given DESC;
```

This identifies:

- Number of reviews submitted by each user
- Average score given by each user

CTE Analysis

```
WITH AnimeStats AS (
```

```
    SELECT
```

```
        genre_id,
```

```
        COUNT(*) AS total_anime,
```

```
        AVG(rating) AS avg_rating
```

```
    FROM anime
```

```
    GROUP BY genre_id
```

```
)
```

```
SELECT
```

```
    g.genre_name,
```

```
    AnimeStats.total_anime,
```

```
    AnimeStats.avg_rating
```

```
FROM AnimeStats
```

```
JOIN genre g
```

```
ON AnimeStats.genre_id = g.genre_id
```

```
ORDER BY AnimeStats.avg_rating DESC;
```

The CTE calculates anime statistics by genre before joining the result with the genre table.

Window Function — Anime Ranking

```
SELECT
```

```
    anime_name,
```

```
    rating,
```

```
RANK() OVER (ORDER BY rating DESC) AS rating_rank
```

```
FROM anime;
```

This ranks anime according to their rating.

Character Power Ranking

```
SELECT
```

```
    character_name,
```

```
    power_level,
```

```
    RANK() OVER (ORDER BY power_level DESC) AS power_rank
```

```
FROM characters;
```

This ranks characters according to their power level.

Database Relationships

Genre → Anime

genre 1 ————— * anime

Studio → Anime

studio 1 ————— * anime

Anime → Characters

anime 1 ————— * characters

Anime → Reviews

anime 1 ————— * reviews

Users → Reviews

users 1 ————— * reviews

Skills Demonstrated

SQL

- Database design

- Relational databases
- Primary keys
- Foreign keys
- Data insertion
- Data filtering
- Aggregation
- JOIN operations
- Subqueries
- CTEs
- CASE statements
- Window functions
- Ranking

Data Analytics

- Anime rating analysis
- Genre analysis
- Studio analysis
- Character analysis
- Popularity analysis
- Review analysis
- User analysis

Project Learning Outcomes

This project demonstrates:

1. Relational database design.
2. SQL table creation.
3. Primary and foreign key relationships.
4. Structured data insertion.
5. Data filtering and sorting.

6. Aggregate calculations.
 7. Multi-table JOIN operations.
 8. Conditional analysis using CASE.
 9. Subquery implementation.
 10. CTE implementation.
 11. Window functions and ranking.
 12. Practical SQL data analysis.
-

Future Improvements

The project can be expanded by adding:

- Anime synopsis
 - Anime images
 - Multiple genres
 - Streaming platforms
 - User watch history
 - Favorite anime
 - Watch status
 - More countries
 - More reviews
 - Review dates
 - Anime popularity rankings
 - Character images
 - Voice actors
 - Manga information
-